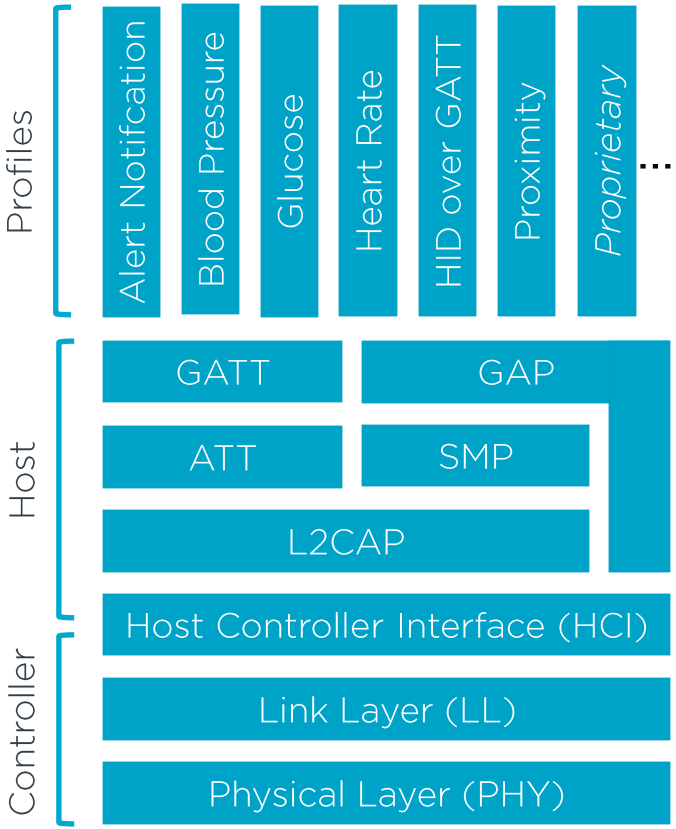
Bluetooh Low Energy

Hands-on LED/Button service (LBS)

© Nordic Semiconductor

Architecture

- Profiles – Services and characteristics
- Host – Connection, discovery, advertising
- Controller – Radio connection



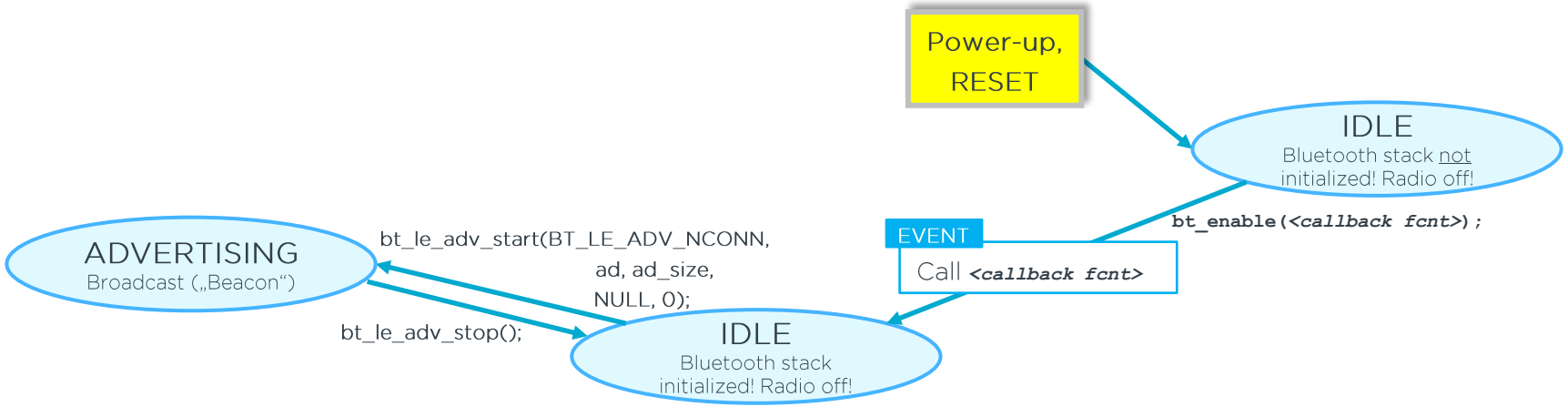
Bluetooth LE

Beacon

© Nordic Semiconductor

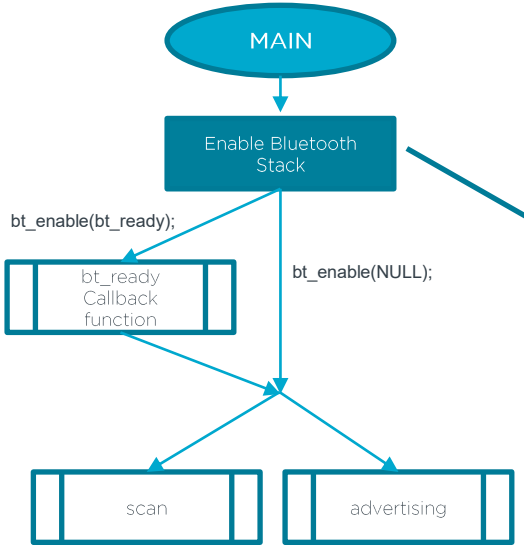
Bluetooth Stack Initialization and Advertising

Event Handler State Diagram



© Nordic Semiconductor

Initialize Bluetooth Stack




```
err = bt_enable(NULL or bt_ready);
if (err) {
   LOG_ERR("Bluetooth init failed (err %d)", err);
   return;
}

LOG_INF("Bluetooth initialized");
```

Starting Advertising: bt_data

Example: iBeacon

```
static const struct bt_data ad[] = {
        BT_DATA_BYTES(BT_DATA_FLAGS, BT_LE_AD_NO_BREDR),
        BT_DATA_BYTES(BT_DATA_MANUFACTURER_DATA,
                      0x4c, 0x00, /* Apple */
                      0x02, 0x15, /* iBeacon */
                      0x18, 0xee, 0x15, 0x16, /* UUID[15..12] */
                      0x01, 0x6b, /* UUID[11..10] */
                      0x4b, 0xec, /* UUID[9..8] */
                      0xad, 0x96, /* UUID[7..6] */
                      0xbc, 0xb9, 0x6d, 0x16, 0x6e, 0x97, /* UUID[5..0] */
                      0x00, 0x00, /* Major */
                      0x00, 0x00, /* Minor */
                      IBEACON_RSSI) /* Calibrated RSSI @ 1m */
};


        /* Start advertising */
        err = bt_le_adv_start(BT_LE_ADV_NCONN, ad, ARRAY_SIZE(ad),
                              NULL, 0);
```

int **bt_le_adv_start**(*const struct* bt_le_adv_param*param, *const struct* bt_data*ad, size_tad_len, *const struct* bt_data*sd, size_tsd_len)

Start advertising.

Set advertisement data, scan response data, advertisement parameters and start advertising.

When the advertisement parameter peer address has been set the advertising will be directed to the peer. In this case advertisement data and scan response data parameters are ignored. If the mode is high duty cycle the timeout will be BT_GAP_ADV_HIGH_DUTY_CYCLE_MAX_TIMEOUT.

Parameters:
- **param** – Advertising parameters.
- **ad** – Data to be used in advertisement packets.
- **ad_len** – Number of elements in ad
- **sd** – Data to be used in scan response packets.
- **sd_len** – Number of elements in sd

Returns: Zero on success or (negative) error code otherwise.

Returns: -ENOMEM No free connection objects available for connectable advertiser.

Returns: -ECONNREFUSED When connectable advertising is requested and there is already maximum number of connections established in the controller. This error code is only guaranteed when using Zephyr controller, for other controllers code returned in this case may be -EIO.

© Nordic Semiconductor

Hands-on: Let‘s make a beacon

Let’s build it

LED and Button Service (LBS)

© Nordic Semiconductor

Nordic Led/Button Service

The Hello World with Bluetooth Low Energy

- Build the Nordic “peripheral_lbs” sample
- Test application using the nRF Connect Mobile apps
- Change the BLE advertising type
- Add handler for connected state

Change the device name to unique name for broadcasting

Modify the name in prj.conf: `CONFIG_BT_DEVICE_NAME=“Your_Name”`

Let’s test it

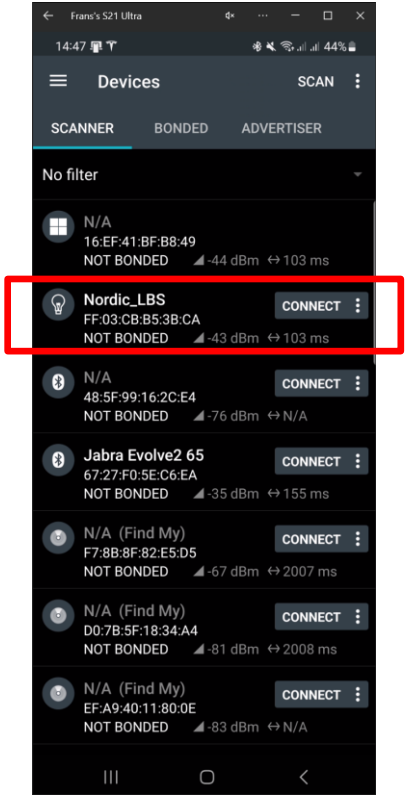
LED and Button Service (LBS)

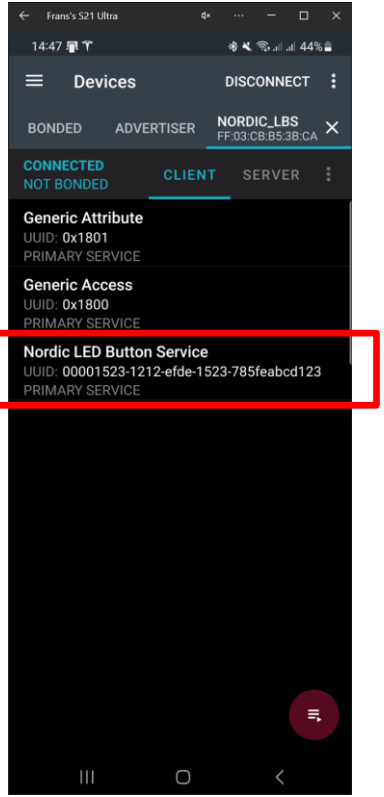
© Nordic Semiconductor

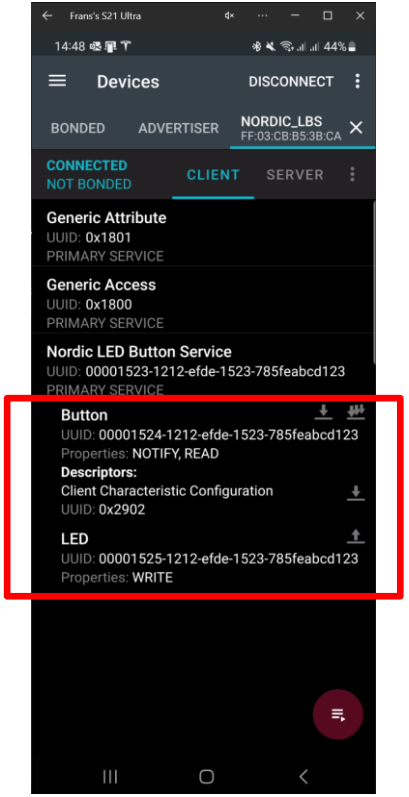
- LED 0:
  Blinks when the main loop is running (that is, the device is advertising) with a period of two seconds, duty cycle 50%.
- LED 1:
  Lit when the development kit is connected.
- LED 2:
  Can be activated/deactivated through a Bluetooth write operation.
- Button 0:
  Send a notification with the button state: "pressed" or "released".

© Nordic Semiconductor

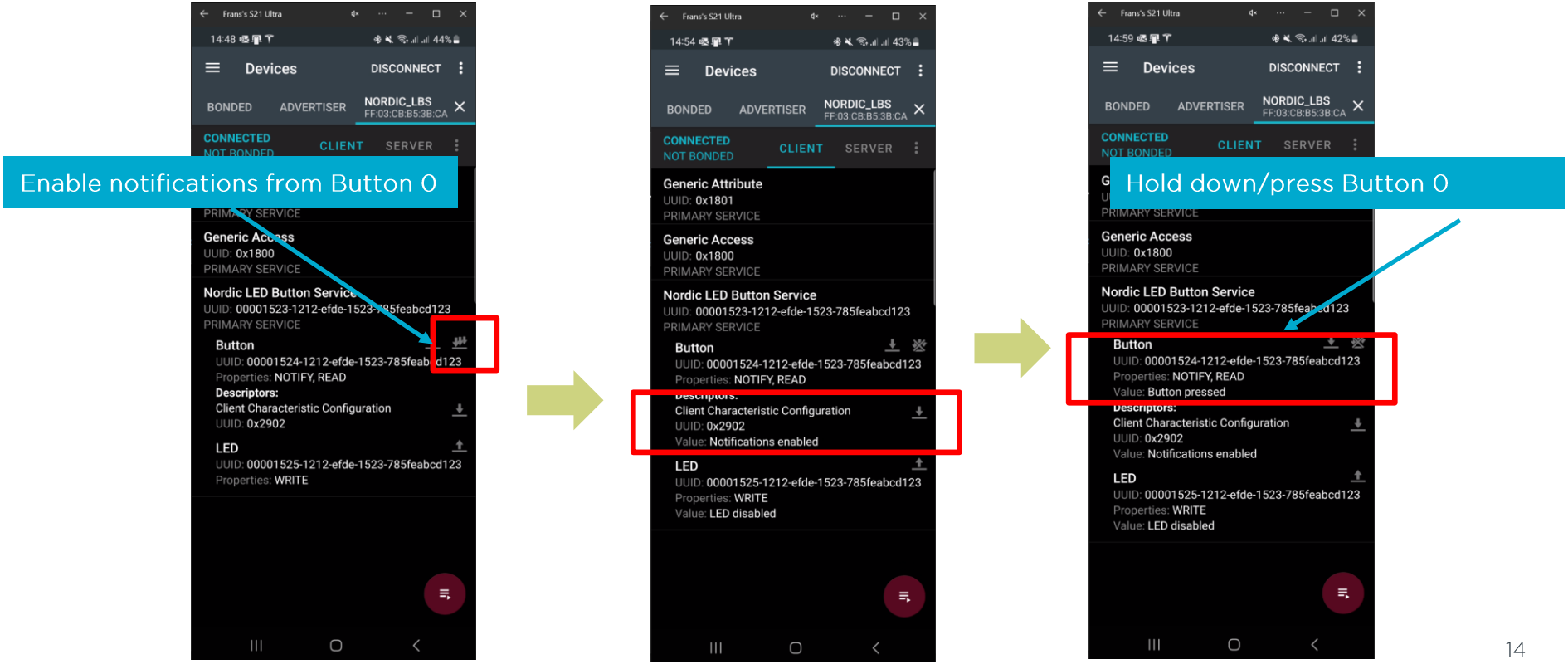
nRF Connect for Mobile



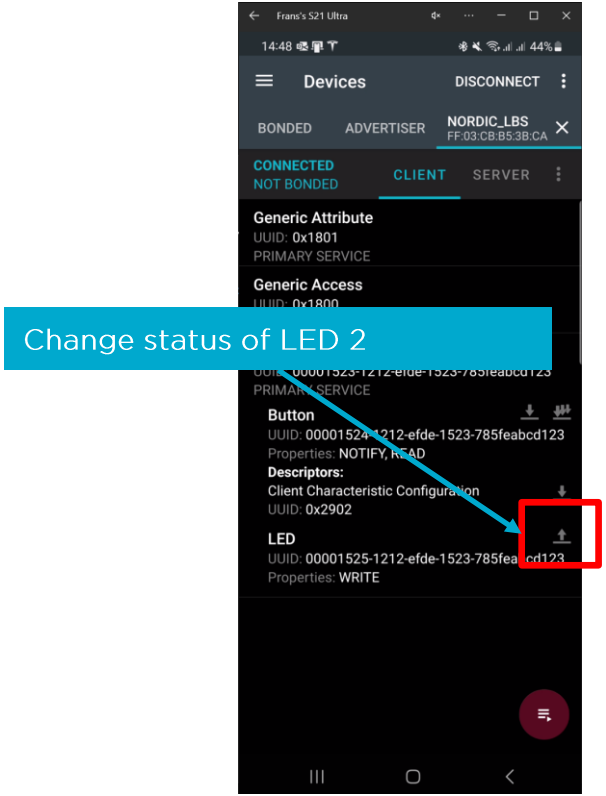




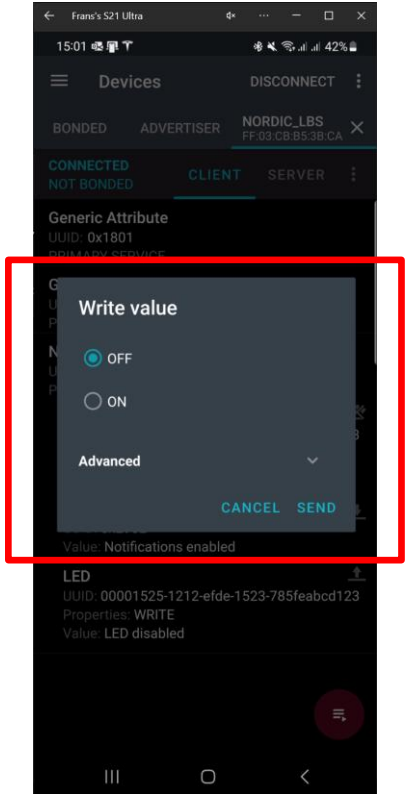
View button state advertisement



Change LED status on development kit



Change status of LED 2

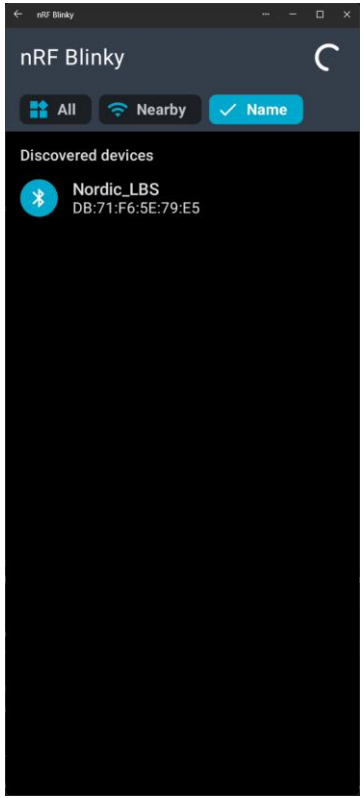


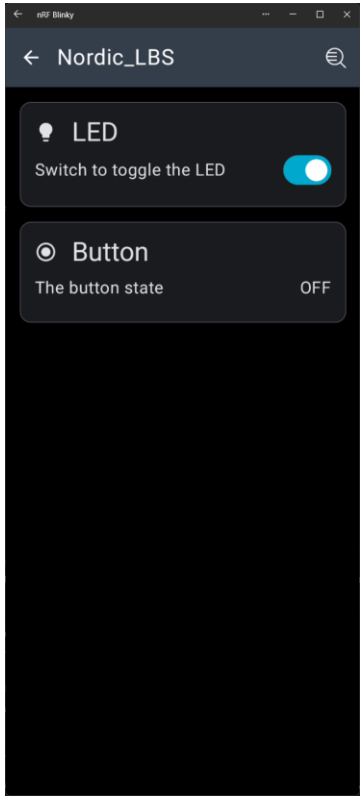
iOS users:

The Nordic LBS service, and specifically its LED state characteristic can be written with ByteArrays or Boolean values:

- 0x00 (off) / bool false
- 0x01 (on) / bool true

Mobile app: nRF Blinky





Analyze the application

Project configuration (file: prj.conf)

Enable Bluetooth

Setup as peripheral device

The name the devices will use during advertisement

```
CONFIG_BT=y
CONFIG_BT_PERIPHERAL=y
CONFIG_BT_DEVICE_NAME="Nordic_LBS"
```

Enable the Nordic Led Button Service

Enable the button feature

```
# Enable the LBS service
CONFIG_BT_LBS=y
CONFIG_BT_LBS_POLL_BUTTON=y
```

Enable the Development Kit library for adding support for the Leds and Buttons/switches

```
CONFIG_DK_LIBRARY=y
```

© Nordic Semiconductor

Advertising Package

- Array with type struct bt_data which will be used to send out data. It consist of the advertising flags and the advertising packet data.

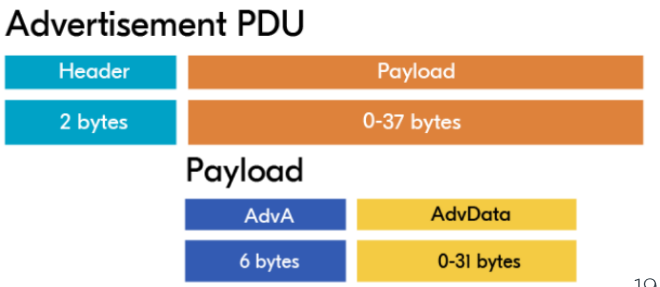
`BT_DATA_BYTES()` Is a macro to generate the header

- `BT_DATA_FLAGS` Indicator of package type
- `BT_LE_AD_GENERAL` Indicate that advertising is available for long time (no time out)
- `BT_LE_AD_NO_BREDR` Classic Bluetooth not supported

`BT_DATA()` Is a macro to generate the payload

- `BT_DATA_NAME_COMPLETE` Indicator of package type
- `DEVICE_NAME` Defined name in prj.conf
- `DEVICE_NAME_LEN` Length

```
43  static const struct bt_data ad[] = {
44      BT_DATA_BYTES(BT_DATA_FLAGS, (BT_LE_AD_GENERAL | BT_LE_AD_NO_BREDR)),
45      BT_DATA(BT_DATA_NAME_COMPLETE, DEVICE_NAME, DEVICE_NAME_LEN),
46  };
```



© Nordic Semiconductor

Scan Response packet

- There are numerous standard data types that can be included in the advertising data (either in the advertising packet or the scan response). These data types are defined in the Bluetooth Supplement to the Core Specification.
- In this example we response with the UUID of the Nordic LBS service
- Like with the advertisement package the Macro BT_DATA_BYTES is used.

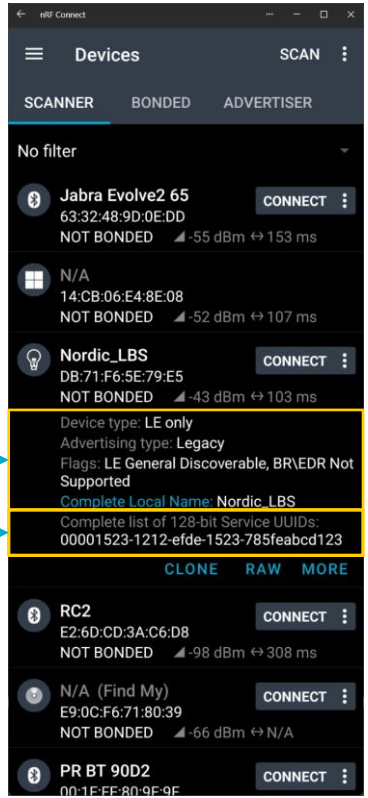
- `BT_DATA_UUID128_ALL` Indicate the data type (Standard BLE UUID)
- `BT_UUID_LBS_VAL` UUID for the Nordic LBS Service

```
static const struct bt_data sd[] = {
    BT_DATA_BYTES(BT_DATA_UUID128_ALL, BT_UUID_LBS_VAL),
};
```

Advertising & Response (Service UUID)

Advertising information

Response package with Service UUID



Define BLE call backs

Call back definition from HOST controller

[connected, disconnected, security_changed,

le_param_req, le_param_updated, le_phy_updated, le_data_len_updated, identity_resolved, remote_info_available]

```
BT_CONN_CB_DEFINE(conn_callbacks) = {
    .connected        = connected,
    .disconnected     = disconnected,
#ifdef CONFIG_BT_LBS_SECURITY_ENABLED
    .security_changed = security_changed,
#endif
};
```

Call back definition for LBS Profile

```
static struct bt_lbs_cb lbs_callbacs = {
    .led_cb    = app_led_cb,
    .button_cb = app_button_cb,
};
```

Main function

Enable Bluetooth

```
err = bt_enable(NULL);
```

Load paired devices information from flash when enable

```
if (IS_ENABLED(CONFIG_SETTINGS)) {
    settings_load();
}
```

Setup the callback programs for the BLE profile (led_cb & button_cb)

```
err = bt_lbs_init(&lbs_callbacs);
```

Start advertising using the define advertising package and scan response package. BT_LE_ADV_CONN defines advertisement type.

```
err = bt_le_adv_start(BT_LE_ADV_CONN, ad, ARRAY_SIZE(ad),
                      sd, ARRAY_SIZE(sd));
```

© Nordic Semiconductor

Buttons and LED Library

Button and Led library for development kits

```
#include <dk_buttons_and_leds.h>
```

Define LEDs for BLE connection status
Blink interval
Define LED which is controlled by user
Define which button is used to indicate status

```
#define RUN_STATUS_LED          DK_LED1
#define CON_STATUS_LED          DK_LED2
#define RUN_LED_BLINK_INTERVAL  1000

#define USER_LED                DK_LED3

#define USER_BUTTON             DK_BTN1_MSK
```

Call back function on button event

```
static void button_changed(uint32_t button_state, uint32_t has_chang
```

Define call back for button events

```
    err = dk_buttons_init(button_changed);
```

Init I/O pins used by LED library

```
    err = dk_leds_init();
```